

Assignment 2- Case Study

Stage 3 Lab Activities

SmartCare Domain Modelling

AI OFF -> AI ON -> COMPARE -> VERIFY | 1 hour

A - Requirements Review

Highlight nouns, verbs and business rules in SmartCare v0.2.

What to highlight	Where
Nouns	Patient, practitioner, appointment, patient information, date, time, status, history
Verbs	Create,view,update,search,link,check,display,cancel,save
Business Rules	One patient and practitioner per booking, Check conflict before saving, approved statuses
Provisional	Cancellation without deletion and history retention, practitioner/date query

B - Candidate Classes

Record candidate concepts, supporting requirements, state and behaviour.

Who	Support	Sate and behavior	Decision
Patient	FR-01 to FR-03	Patient_id, name; update_details	Keep. Search is a separate collection query.
Practitioner	FR-04,FR-05	Practitioner_id, name; update_details(name)	Keep. Schedule query remains provisional
Appointment	FR-05 to FR-10; update scope	ID, links, time, status; change_status(), proposed reschedule(), provisional cancel().	Keep. Confirm rules before implementing.
Name / Status	FR-02, FR-04, FR-08	Details belonging to another object.	Attributes, not independent entities

C - CRC Cards

Create CRC cards for Patient, Practitioner and Appointment.

Patient

Responsibilities	Collaborators
------------------	---------------

Hold identity and patient details; update its own details after validation	No domain collaborator is required to edit its own name
Participate as the patient linked to each appointment (FR-05)	Appointment. Querying the full history belongs to a later application/repository layer

Practitioner

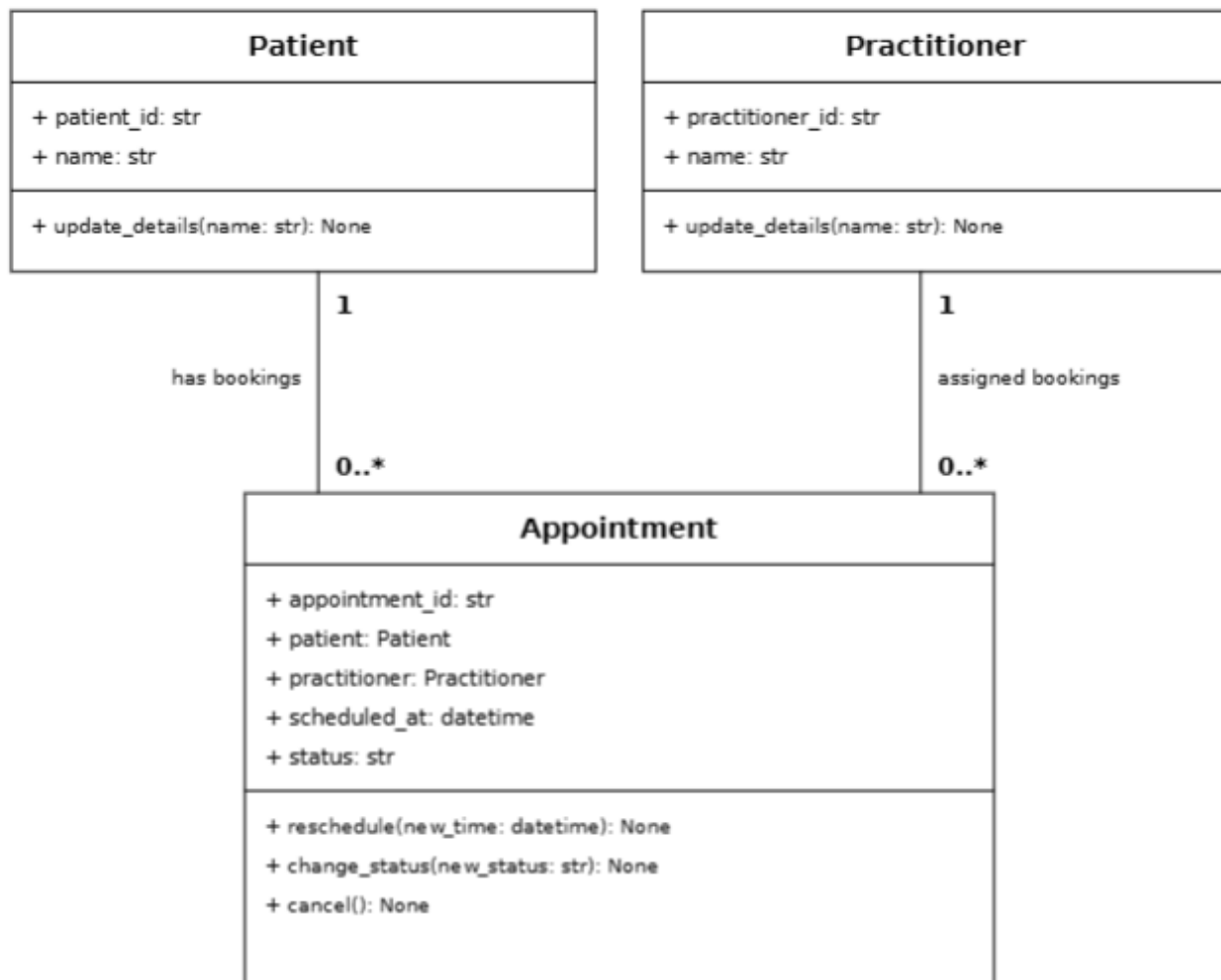
Responsibilities	Collaborators
Hold identity and practitioner details; update its own details after validation (FR-04)	No domain collaborator is required to edit its own name.
Participate as the practitioner assigned to an appointment (FR-05). Its identity can support a schedule query	Appointment. Schedule collection and filtering are deferred to an application/repository layer

Appointment

Responsibilities	Collaborators
Hold booking identity, date/time and current status (FR-07); change status using approved rules (FR-08).	Patient and Practitioner, referenced by this appointment
Keep its patient and practitioner links (FR-05); expose proposed reschedule() and provisional cancel() operations	A future booking service must coordinate cross-record conflict checks (FR-06) (out of core model)

D - UML Model

Draw classes, attributes, operations, associations and multiplicities.



E - AI Design Review

Ask AI to suggest classes and relationships using only confirmed requirements; require supporting requirement IDs.

Act as a software domain-model reviewer. Use only the confirmed SmartCare v0.2 requirements when recommending classes and relationships. Cite supporting requirement IDs for every suggestion. Distinguish a domain entity from an attribute, operation, application service or storage mechanism. Review responsibilities, multiplicities and model-code consistency. Flag provisional details and unsupported features; do not invent business rules.

Review findings

Patient, Practitioner and Appointment are justified by the brief and FR-01, FR-04 and FR-05. Status belongs to Appointment (FR-08). No requirement supports notifications, a Clinic ownership hierarchy or a schedule optimisation engine. FR-06 requires comparison with other bookings, so a single Appointment should not be responsible for searching all bookings

F - Compare and Decide

Record at least one accepted, modified and rejected AI suggestion.

Give Appointment patient and practitioner references.

Evidence: FR-05 links one patient and one practitioner in the draft.

Decision: Accepted

Reason: Keeps relationship visible and stops patient name being copying in every booking.

Model change: Add patient: Patient and practitioner: Practitioner; use A2 multiplicities.

Use AppointmentManager to check conflicts, save and update booking state.

Evidence: FR-06 supports conflict checks; FR-08 supports status changes

Decision: Modified

Reason: Separate single-booking state from cross-record coordination. A generic manager is too broad.

Model change: Keep local operations in Appointment; defer a focused service and repository.

Add NotificationManager and SMS reminders.

Evidence: No supporting requirement; reminders are out of scope in v0.2.

Decision: Rejected

Reason: It adds an unrequested integration and does not solve a confirmed modelling need.

G - Python Skeletons

Create simple Patient, Practitioner and Appointment class skeletons.

Whole different python file

H - Consistency Check

Check	Results
Class names	Patient, Practitioner and Appointment appear in both UML and Python
Attributes and types	All names and types match
Scope and assumptions	No reminders, payments, Clinic composition, permanent retention or invented status enumeration
Skeleton boundary	No full business behaviour or persistence is implemented

Reflection

What modelling decision was hardest? Where did AI over-design? What evidence supported your final choices?

The hardest decision was which nouns should become classes. Name, status and cancellation initially looked like possible classes, but they do not need independent identities in this small model. I represented name and status as attributes and cancellation as a provisional Appointment operation. Additionally, I had to distinguish between tasks that could be completed in a single appointment and those that needed access to every appointment, such looking for conflicting reservations. AI might overdesign the solution by proposing distinct managers for each item, for as a Schedule Engine, a Notification Manager, or a Clinic class that owns every object. Because the brief and FR-01, FR-04, and FR-05 specifically support those concepts, I decided to stick

with Patient, Practitioner, and Appointment. The final decisions are supported by clearly stated assumptions and traceable needs. Status values, conflict rules, and cancelation retention still require confirmation. The Python skeleton and the UML might be compared to ensure that class names, attributes, references, and operation signatures agree without implementing unapproved behavior.